A stable master is an important feature of a healthy cluster. If any ping from the nodes returns a negative response (meaning the master is probably down!), the nodes do not wait for another timed call but instead call for an election immediately. The master is elected instantly from the list of master-eligible nodes.

15.3.2 Unassigned shards

When we create an index with a set number of shards, Elasticsearch works out a strategy to allocate shards to available nodes. In a hypothetical example of 5 nodes with an index having 10 shards, each node is allocated 2 shards to balance shard assignment. Replicas are balanced similarly, although they are never allocated on the same node as their partners.

In some instances, shards cannot be assigned. For example, if we have a newly created index with 10 shards on a 5-node cluster and one or all of the shards are unassigned (unallocated) to any of these 5 nodes, the cluster will complain about unassigned shards.

However, we are talking about a hypothetical situation where one or all shards aren't allocated to any of the nodes. Unassigned or unallocated shards are a production incident. This usually happens during rebalancing of the shards due to a node (or a few nodes) failing for whatever reason. The unassigned shards have operational consequences on indexes that are newly created (fresh indexes) or already live after being created earlier.

If shards are unassigned for an *existing* index (most likely during the rebalancing phase after one or a few nodes die), read operations (searches) are halted because that shard's share of documents may have disappeared. Write operations also halt because of the same problem of unassigned shards. If this is true for a newly created index with unassigned shards, write operations pause until the shard allocation is fixed.

Elasticsearch provides a convenient `allocation` API to check the unassigned nature of shards. Issuing the following command fetches a detailed explanation of why Elasticsearch can't assign the shards to relevant nodes.

Listing 15.12 Getting an explanation for unassigned shards

```
GET _cluster/allocation/explain
```

Because we didn't specify any shards with this request, Elasticsearch chooses an unassigned shard randomly and explains why the allocation wasn't successful. For example, on my single-node development cluster, invoking this command produces the response shown in figure 15.1.

There's a lot of information to digest in the output of the allocation explanation call. Figure 15.1 shows that the `programming_books2` index is unassigned (look at the `current_state` attribute). The `allocate_explanation` attribute explains the reason